



LAB ASSIGNMENT -4

OPERATING SYSTEM (UCS-303)

Name –Harvinder singh

Course –MCA (26-28)

Roll no - 2026010044

Q1. Write a program to implement fork() system call.

Answer

The fork() system call is used to create a new process. The new process is called the child process, while the process that creates it is called the parent process. After fork() returns, both processes continue execution from the next statement.

C Program

```
#include <stdio.h>
#include <unistd.h>
```

```
int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
    }
    else if (pid == 0)
    {
        printf("This is the child process.\n");
        printf("Child Process ID: %d\n", getpid());
    }
    else
    {
        printf("This is the parent process.\n");
        printf("Parent Process ID: %d\n", getpid());
        printf("Child Process ID: %d\n", pid);
    }

    return 0;
}
```

Explanation

If fork() returns a negative value, process creation has failed. If it returns 0, the program is executing in the child process. If it returns a positive value, the program is executing in the parent process and the returned value is the child's process ID.

Q2. Write a program to implement wait() and exit() System Calls.

Answer

The wait() system call makes the parent process wait until one of its child processes terminates. The exit() system call terminates the calling process and returns a status value to the operating system.

C Program

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>
```

```

#include<stdlib.h>

int main()
{
    pid_t pid;

    pid = fork();

    if (pid < 0)
    {
        printf("Fork failed.\n");
        return 1;
    }
    elseif(pid== 0)
    {
        printf("Child process is running.\n");
        printf("Child process is exiting.\n");
        exit(0);
    }
    else
    {
        printf("Parent is waiting for child...\n");
        wait(NULL);
        printf("Child process has terminated.\n");
        printf("Parent process continues.\n");
    }

    return 0;
}

```

Explanation

The child process prints its messages and terminates using exit(0). The parent process calls wait(NULL), so it waits for the child to finish before continuing its execution.

Q3. Write a program to implement execv() system call.

Answer

The execv() system call is used to replace the current process image with a new program. It does not create a new process; instead, the calling process is replaced by the program specified in execv().

C Program

```

#include <stdio.h>
#include <unistd.h>

```

```

int main()
{
    char *args[] = {"ls", "-l", NULL};

    printf("Executing ls command using execv()...\n");
}

```

```

execv("/bin/ls", args);

printf("execv() failed.\n");

return 1;
}

```

Explanation

The program creates an argument list for the ls command and passes it to execv(). If execv() succeeds, the current program is replaced by the ls program and the statement after execv() is not executed. If execv() fails, the program prints an error message.

Q4. Write a program to implement the system calls open(), read(), write() & close().

Answer

These system calls are used for basic file management. open() opens or creates a file, read() reads data from a file, write() writes data to a file, and close() closes an opened file.

C Program

```

#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <string.h>

int main()
{
    int fd;
    char buffer[100];
    const char *text = "Operating System System Calls";

    fd = open("sample.txt", O_CREAT | O_RDWR | O_TRUNC, 0644);

    if (fd < 0)
    {
        printf("File could not be opened.\n");
        return 1;
    }

    write(fd, text, strlen(text));

    lseek(fd, 0, SEEK_SET);

    int n = read(fd, buffer, sizeof(buffer) - 1);

    if (n < 0)
    {
        printf("File could not be read.\n");
        close(fd);
        return 1;
    }
}

```

```
}  
  
buffer[n] = '\0';  
  
printf("Data read from file: %s\n", buffer);  
  
close(fd);  
  
return 0;  
}
```

Explanation

First, open() creates or opens sample.txt with read and write access. The write() call stores text in the file. lseek() moves the file position back to the beginning so that read() can read the newly written data. Finally, close() releases the file descriptor.